DEPARTMENT OF
COMPUTER SCIENCE & ENGINEERING
Discover. Learn. Empower.

Experiment No.: 7

Student Name: Probal Dhali
UID: 24BAI71013
Branch: AIT-CSE
Section/Group: B
Semester: 5TH
Date of Performance:
Subject Name: Competitive Coding – II
Subject Code: 24CSP-305

Experiment No: 7.1

Aim: To write a C++ program to repeatedly add the digits of a number until a single-digit number is obtained.

Objective: To understand digit manipulation and implement the Add Digits problem using C++.

Problem Statement: Given an integer num, repeatedly add all its digits until the result has only one digit, and return that digit.

LeetCode Link: https://leetcode.com/problems/add-digits/description/?utm_source=chatgpt.com

Procedure/Algorithm/Pseudocode:

Start.
Take an integer num as input.
While num is greater than or equal to 10:

- Initialize sum = 0.
- Extract each digit using num % 10.
- Add the digit to sum.
- Remove the last digit using num / 10.
- Assign sum to num.

Return num.
Stop.

Code:

```
class Solution {

public:

   int addDigits(int num) {

      while (num >= 10) {

         int sum = 0;

         while (num > 0) {

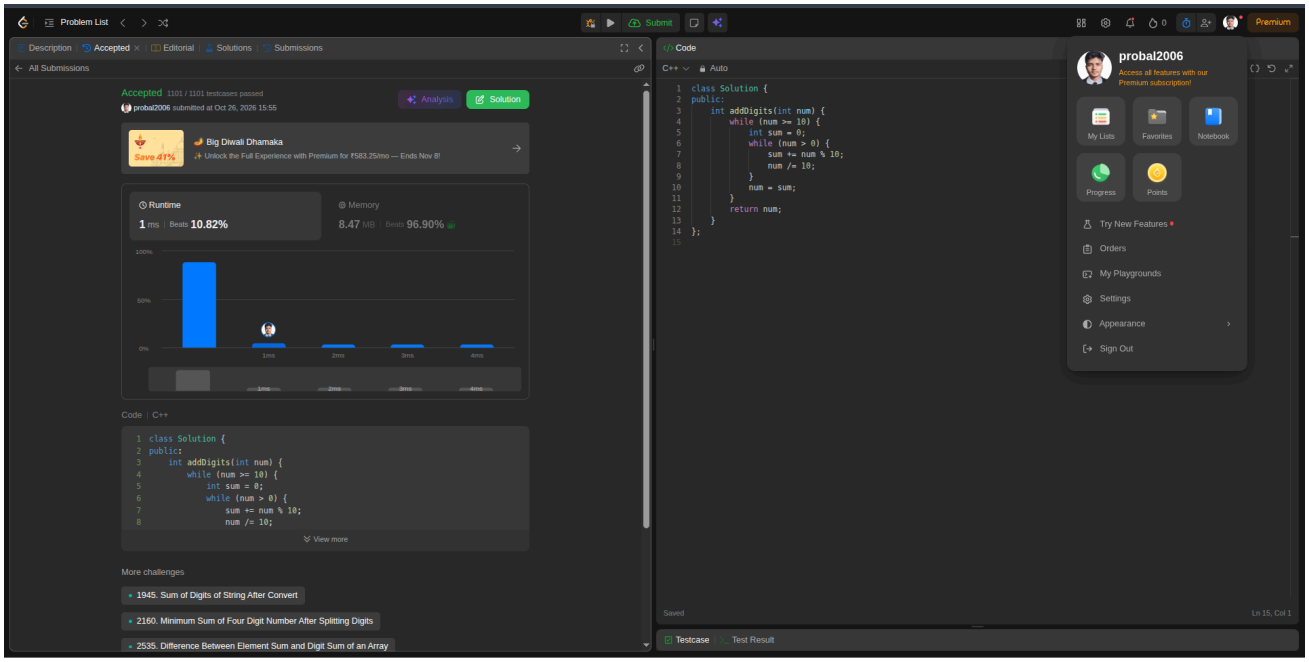
            sum += num % 10;

            num /= 10;

         }
```

```
            num = sum;
        }
        return num;
    }
};
```

Output:



Observations/Outcome: The program successfully calculates the digital root by repeatedly adding the digits until a single-digit number is obtained.

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Learning outcomes :

- Understand digit extraction using % and /.
- Implement loops for repeated digit addition.
- Develop problem-solving skills using C++.
- Analyze time and space complexity.

Experiment No: 7.2

Aim: To write a C++ program to find the duplicate number in an array.

Objective: To understand array manipulation and implement an efficient algorithm to detect duplicate elements.

Problem Statement: Given an array nums containing n + 1 integers, where each integer is in the range [1, n], find the duplicate number without modifying the array and using only constant extra space.

LeetCode Link: https://leetcode.com/problems/find-the-duplicate-number/description/?utm_source=chatgpt.com

Procedure/Algorithm/Pseudocode:

1. Initialize two pointers, slow and fast, to the first element.
2. Move slow one step and fast two steps at a time.
3. Continue until both pointers meet.
4. Reset slow to the first element.
5. Move both pointers one step at a time until they meet again.
6. Return the meeting value as the duplicate number.

Code:

```
class Solution {

public:

   int findDuplicate(vector<int>& nums) {

      int slow = nums[0];

      int fast = nums[0];

      // Find the intersection point

      do {

         slow = nums[slow];

         fast = nums[nums[fast]];

      } while (slow != fast);

      // Find the entrance to the cycle

      slow = nums[0];

      while (slow != fast) {

         slow = nums[slow];

         fast = nums[fast];

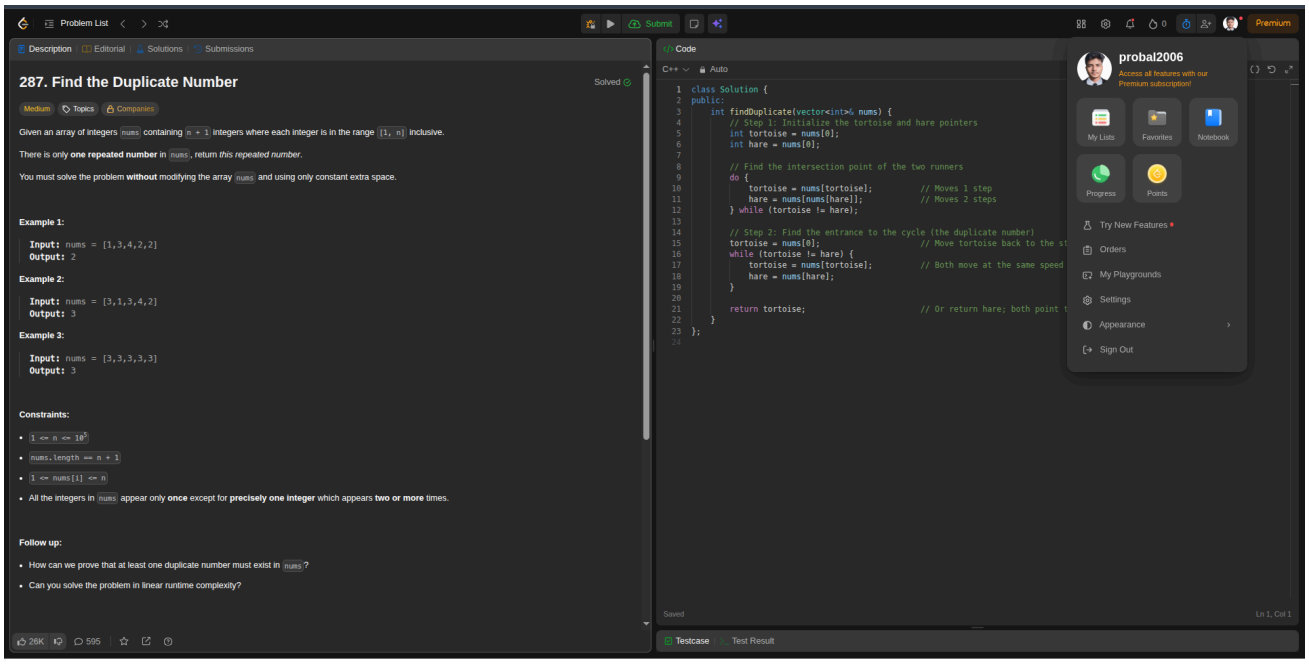
      }
```

```
        return slow;

    }

};
```

Output:



Observations/Outcome: The program successfully identifies the duplicate number using Floyd's Tortoise and Hare cycle detection algorithm without modifying the array.

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Learning outcomes :

- Understand duplicate detection in arrays.
- Learn Floyd's cycle detection algorithm.
- Implement an efficient solution using two pointers.
- Analyze time and space complexity.